Toolformer: Language Models Can Teach Themselves to Use Tools

Timo Schick Jane Dwivedi-Yu Roberto Dessì† Roberta Raileanu Maria Lomeli Eric Hambro Luke Zettlemoyer Nicola Cancedda Thomas Scialom
FAIR, Meta

NeurIPS 2023, Cited: 8510
Reviewed by 허성준

ChatGPT Limitations

Who is the current President of the United States?



As an AI language model, I don't have real-time data or internet access, and my knowledge was last updated in September 2021. [...]

What day of the week is it today?



As an AI language model, I do not have the capability to access current time or date information. [...]

What is the result of 3435 * 235 / 9?



This is approximately 89,937.22.

True answer: 89691.66

ChatGPT Limitations

- Inability to access up-to-date information
- Tendancy of fact hallucinations
- Difficulties in understanding low-resource languages
- Lack of mathematical skills
- Unawareness of the progression of time

Toolformer Tools/APIs

What other name is Pittsburgh known by?



The Steel City

War memorial Flodden



[...] was created in memory of the Battle of Flodden.

3435 * 235 / 9



89691.67

Ø



Thursday, March 10, 2019

Os Melhores Escolas em Jersey



The Best Schools in Jersey

Toolformer Tools/APIs

Table 1: Examples of inputs and outputs for all APIs used.

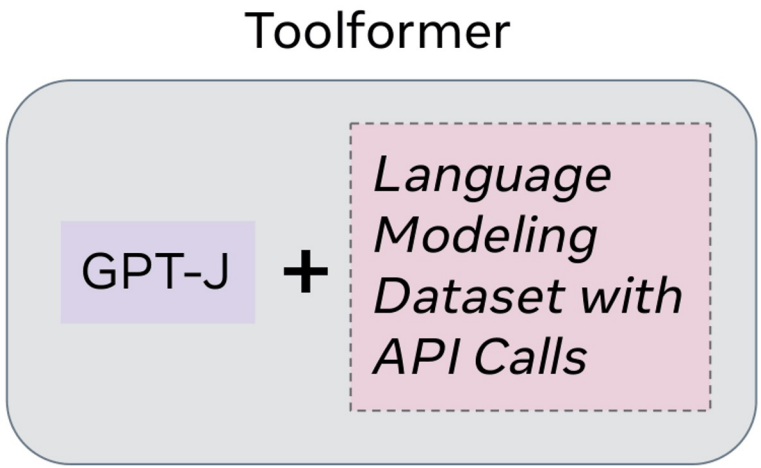
API Name	**Example Input**	**Example Output**
Question Answering	Where was the Knights of Columbus founded?	New Haven, Connecticut
Wikipedia Search	Fishing Reel Types	Spin fishing > Spin fishing is distinguished between fly fishing and bait cast fishing by the type of rod and reel used. There are two types of reels used when spin fishing, the open faced reel and the closed faced reel.
Calculator	27 + 4 * 2	35
Calendar	ε	Today is Monday, January 30, 2023.
Machine Translation	sûreté nucléaire	nuclear safety

Goal of the Research

- Tool use should be learned in a self-supervised way without large amounts of *human annotations*. This is important not only because of the costs associated with such annotations, but also because what humans find useful may be different from what a model finds useful.
- The LM should not lose any of its *generality* and should be able to decide for itself *when* and *how* to use which tool. In contrast to existing approaches, this enables a much more comprehensive use of tools that is not tied to specific tasks.

Steps to Creating Toolformer

1. Creating a new training dataset augmented with API calls
2. Finetune GPT-J using this new dataset



Dataset Examples

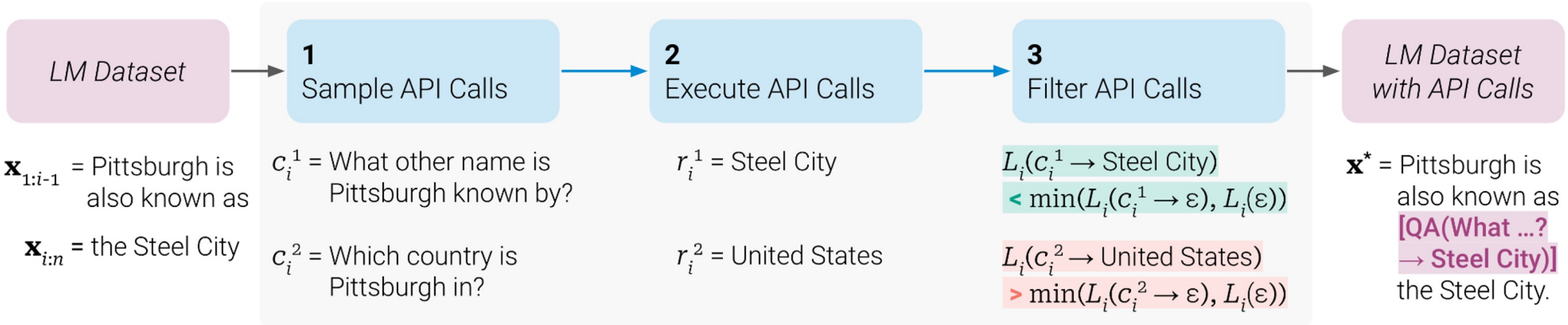
The New England Journal of Medicine is a registered trademark of **[QA("Who is the publisher of The New England Journal of Medicine?") → Massachusetts Medical Society]** the MMS.

Out of 1400 participants, 400 (or **[Calculator(400 / 1400) → 0.29]** 29%) passed the test.

The name derives from "la tortuga", the Spanish word for **[MT("tortuga") → turtle]** turtle.

The Brown Act is California's law **[WikiSearch("Brown Act") → The Ralph M. Brown Act is an act of the California State Legislature that guarantees the public's right to attend and participate in meetings of local legislative bodies.]** that requires legislative bodies, like city councils, to hold their meetings open to the public.

Creating the Training Dataset



Approach

Our aim is to equip a language model M with the ability to use different tools through API calls. We represent API calls as tuples $c = (a_c, i_c)$ where a_c is the name of the API and i_c is the corresponding input. Given an API call c with a corresponding result r, we denote the linearized sequences of the API call not including and including its result, respectively, as:

$$\mathrm{e}(c) = \texttt{<API>}\, a_c(i_c)\, \texttt{</API>} \qquad \mathrm{e}(c, r) = \texttt{<API>}\, a_c(i_c) \rightarrow r\, \texttt{</API>}$$

where "<API>", "</API>" and "→" are special tokens.[1] Some examples of linearized API calls inserted into text sequences are shown in Figure 1.

Sampling API Calls

Sampling API Calls For each API, we write a prompt $P(\mathbf{x})$ that encourages the LM to annotate an example $\mathbf{x} = x_1, \ldots, x_n$ with API calls. An example of such a prompt for a question answering tool is shown in Figure 3. Let $p_M(z_{n+1} \mid z_1, \ldots, z_n)$ be the probability that M assigns to token z_{n+1} as a continuation for the sequence $z_1, \ldots, z_n$. We first sample up to k candidate *positions* for doing API calls by computing, for each $i \in \{1, \ldots, n\}$, the probability $p_i = p_M(\texttt{<API>} \mid P(\mathbf{x}), x_{1:i-1})$ that M assigns to starting an API call at position i. Given a sampling threshold τ_s, we keep all positions $I = \{i \mid p_i > \tau_s\}$; if there are more than k such positions, we only keep the top k. For each position $i \in I$, we then obtain up to m API calls $c_i^1, \ldots, c_i^m$ by sampling from M given the sequence $[P(\mathbf{x}), x_1, \ldots, x_{i-1}, \texttt{<API>}]$ as a prefix and `</API>` as an end-of-sequence token.

Prompting the Model to Generate API Calls

Your task is to add calls to a Question Answering API to a piece of text. The questions should help you get information required to complete the text. You can call the API by writing "[QA(question)]" where "question" is the question you want to ask. Here are some examples of API calls:

Input: Joe Biden was born in Scranton, Pennsylvania.
Output: Joe Biden was born in [QA("Where was Joe Biden born?")] Scranton, [QA("In which state is Scranton?")] Pennsylvania.

Input: *${input}*
Output:

Examples of Generated API Calls

Your task is to add calls to a QA API to a piece of text [...]

Input: Pittsburgh is known as the Steel City.

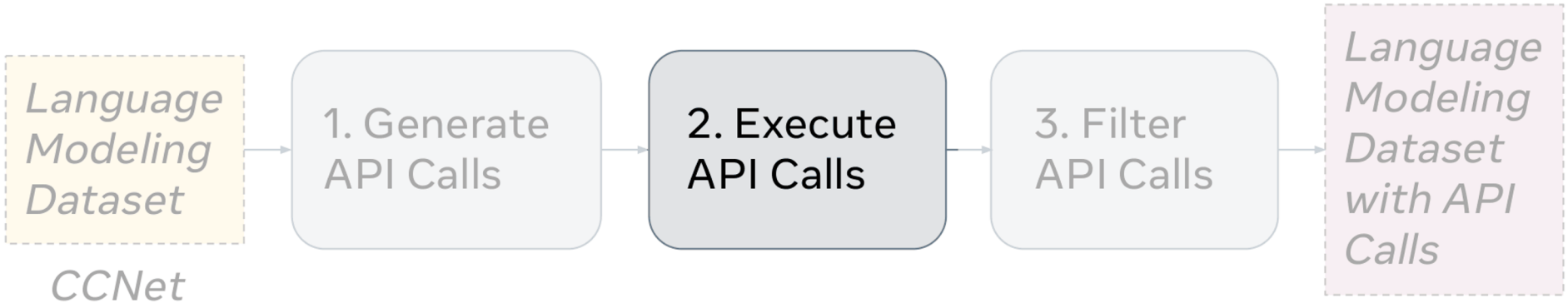
Output:

Pittsburgh is known as [QA("In which state is Pittsburgh?")] the Steel City.

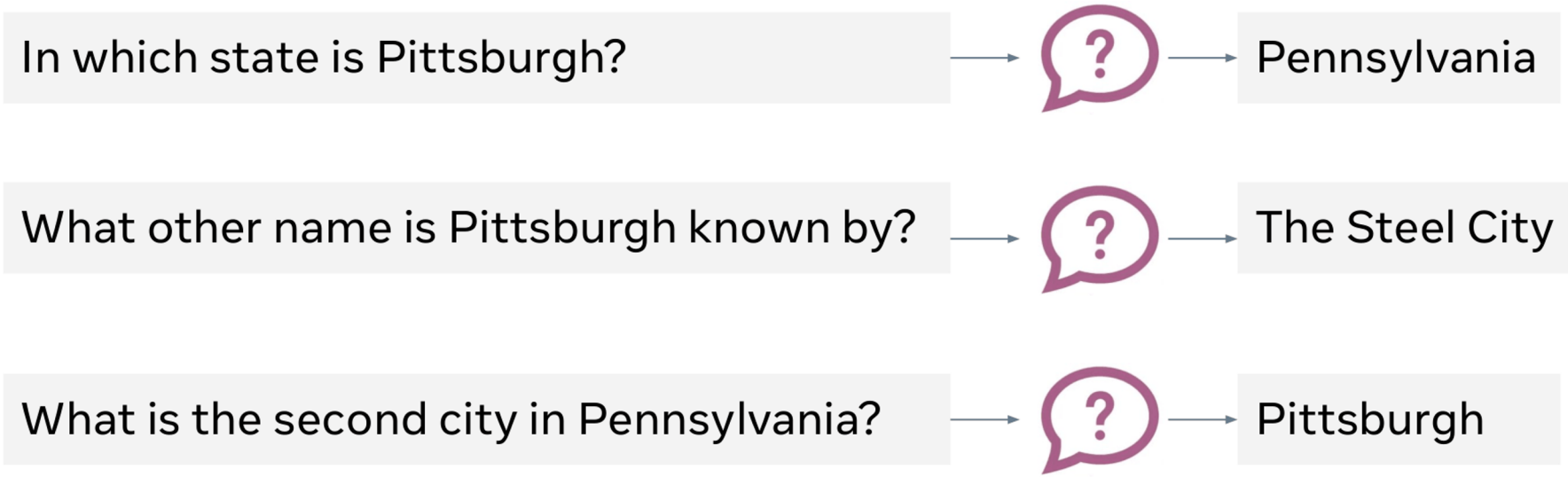
Pittsburgh is known as [QA("What other name is Pittsburgh known by?")] the Steel City.

Pittsburgh is known as [QA("What is the second city in Pennsylvania?")] the Steel City.

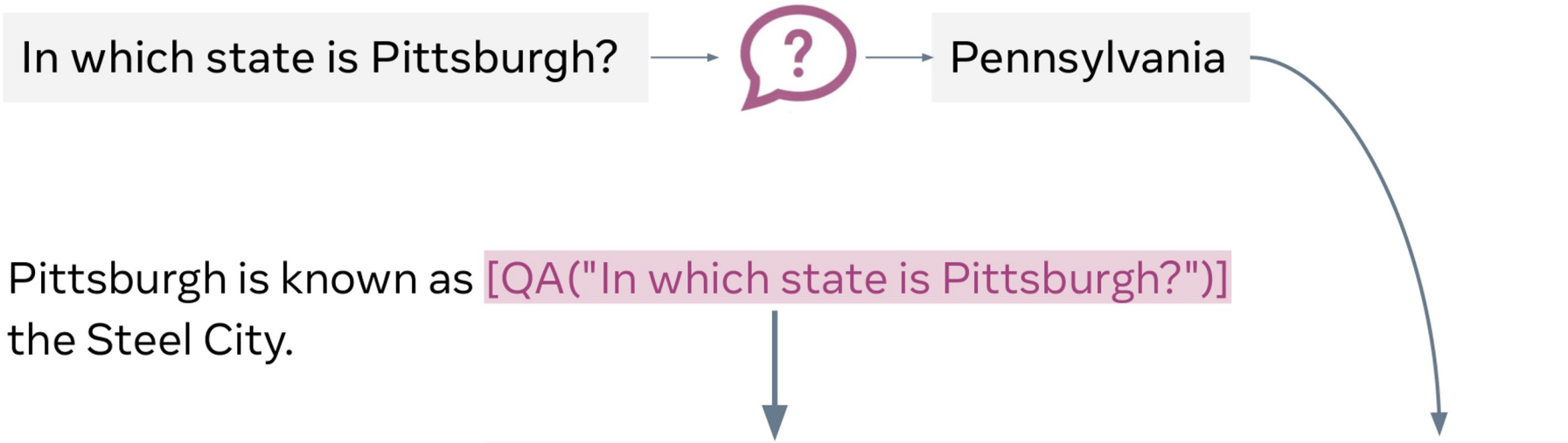
Creating the Training Dataset



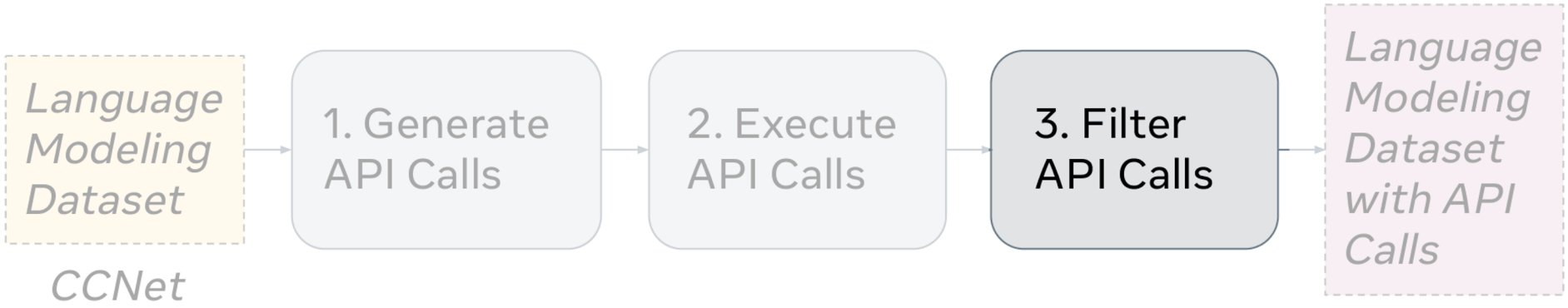
Execute the API Calls



Execute the API Calls



Creating the Training Dataset



Filtering API Calls

Filtering API Calls Let i be the position of the API call c_i in the sequence $\mathbf{x} = x_1, \ldots, x_n$, and let r_i be the response from the API. Further, given a sequence $(w_i \mid i \in \mathbb{N})$ of *weights*, let

$$L_i(\mathbf{z}) = -\sum_{j=i}^{n} w_{j-i} \cdot \log p_M(x_j \mid \mathbf{z}, x_{1:j-1})$$

be the weighted cross entropy loss for M over the tokens $x_i, \ldots, x_n$ if the model is prefixed with some text sequence $\mathbf{z}$. We compare two different instantiations of this loss:

$$L_i^+ = L_i(\mathrm{e}(c_i, r_i)) \qquad\qquad L_i^- = \min\left(L_i(\varepsilon), L_i(\mathrm{e}(c_i, \varepsilon))\right)$$

Filter the API Calls using Model-based Perplexity

$L_\bullet(PREFIX) = -\log p(\text{the Steel City.} \mid PREFIX)$

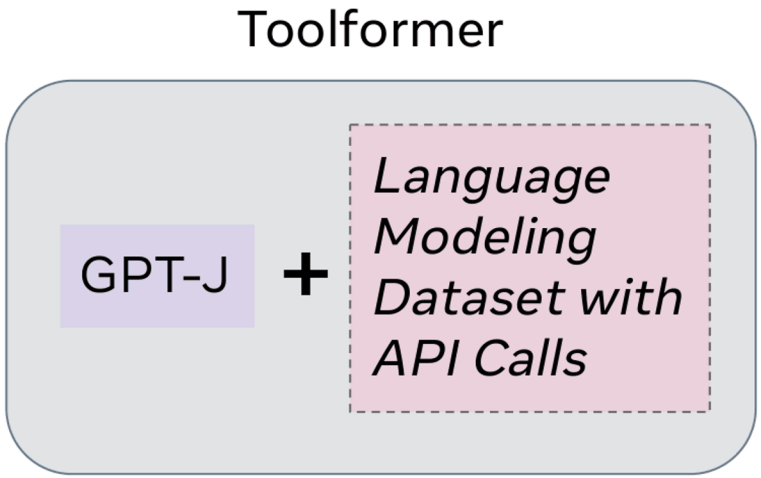
A. No API Call — $L_A(\textit{Pittsburgh is known as}) = \mathbf{2.5}$

B. Non-executed API Call — $L_B(\textit{Pittsburgh is known as [QA("What other name is Pittsburgh known by?") → ?]}) = \mathbf{2.1}$

C. Executed API Call — $L_C(\textit{Pittsburgh is known as [QA("What other name is Pittsburgh known by?") → Steel City]}) = \mathbf{0.8}$

$$\text{Usefulness} = \min(L_A, L_B) - L_C = \min(2.5, 2.1) - 0.8 = 1.3$$

Fine-tuning Toolformer



Evaluation Tasks

1. Fact Completion/Question Answering
    a. “The theory of relativity was developed by _____”
    b. “In Greek Mythology, who is the goddess of spring growth?”
2. Math Computations
3. Multilingual Questions
    a. Context is given in English, question is multilingual.
4. Temporal Questions
    a. How many days is it until Christmas?

Table 3: Results on subsets of LAMA and various benchmarks requiring mathematical reasoning. For LAMA, Toolformer uses the question answering tool for most examples, clearly outperforming all baselines of the same size and achieving results competitive with GPT-3. For the math benchmarks, Toolformer makes extensive use of the calculator tool, clearly outperforming OPT and GPT-3. Best results with a GPT-J based model are shown in bold, best results overall are underlined.

	LAMA			**Math Benchmarks**		
Model	**SQuAD**	**Google-RE**	**T-REx**	**ASDiv**	**SVAMP**	**MAWPS**
GPT-J	17.8	4.9	31.9	7.5	5.2	9.9
GPT-J + CC	19.2	5.6	33.2	9.6	5.0	9.3
Toolformer (disabled)	22.1	6.3	34.9	14.8	6.3	15.0
Toolformer	**33.8**	**11.5**	**53.5**	**40.4**	**29.4**	**44.0**
OPT (66B)	21.6	2.9	30.1	6.0	4.9	7.9
GPT-3 (175B)	26.8	7.0	39.8	14.0	10.0	19.8

Table 4: Results for various question answering datasets and temporal datasets. Using the Wikipedia search tool for most examples, Toolformer clearly outperforms baselines of the same size, but falls short of GPT-3 (175B) for question answering tasks. For temporal datasets, Toolformer outperforms all baselines, but does not make use of the calendar tool for TEMPLAMA.

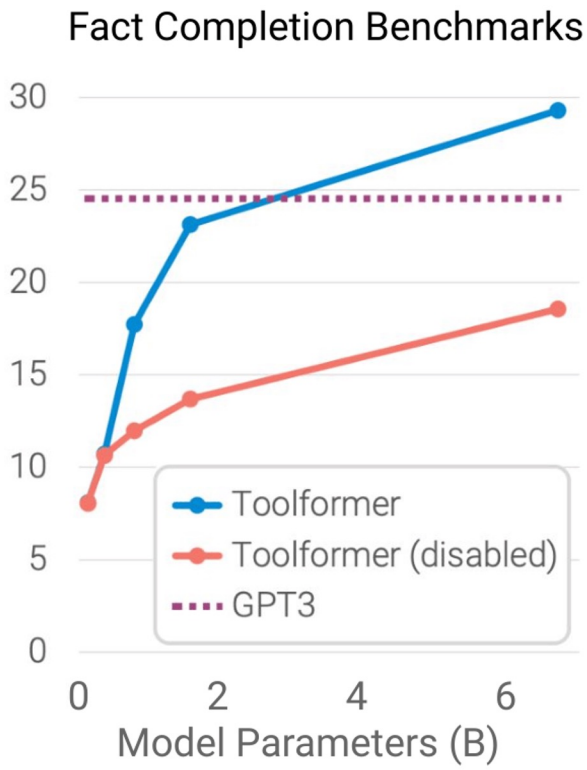
	LAMA			Temporal Datasets	
Model	**WebQS**	**NQ**	**TriviaQA**	**TEMPLAMA**	**DATESET**
GPT-J	18.5	12.8	43.9	13.7	3.9
GPT-J + CC	18.4	12.2	45.6	12.9	2.9
Toolformer (disabled)	18.9	12.6	46.7	12.7	5.9
Toolformer	**26.3**	**17.7**	**48.8**	**16.3**	**27.3**
OPT (66B)	18.6	11.4	45.7	14.5	1.3
GPT-3 (175B)	29.0	22.6	65.9	15.5	0.8

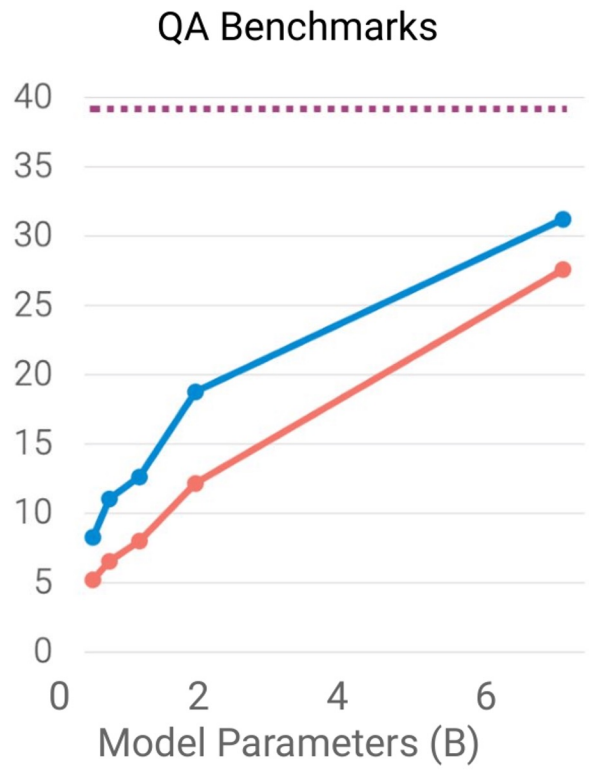
Table 5: Results on MLQA for Spanish (Es), German (De), Hindi (Hi), Vietnamese (Vi), Chinese (Zh) and Arabic (Ar). While using the MT tool to translate questions is helpful across all languages, further pretraining on CCNet deteriorates performance; thus, Toolformer does not consistently outperform GPT-J. The final rows correspond to models that are given contexts and questions in English.

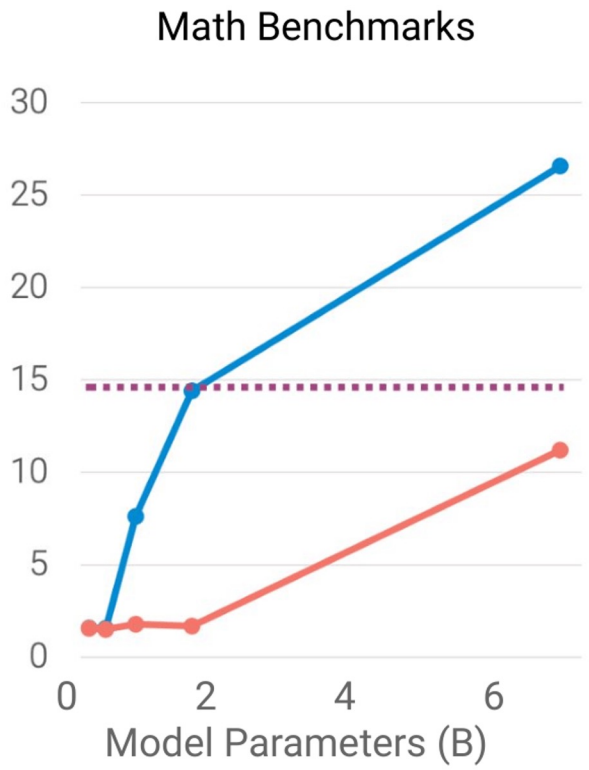
Model	**Es**	**De**	**Hi**	**Vi**	**Zh**	**Ar**
GPT-J	15.2	**16.5**	1.3	8.2	**18.2**	**8.2**
GPT-J + CC	15.7	14.9	0.5	8.3	13.7	4.6
Toolformer (disabled)	19.8	11.9	1.2	10.1	15.0	3.1
Toolformer	**20.6**	13.5	**1.4**	**10.6**	16.8	3.7
OPT (66B)	0.3	0.1	1.1	0.2	0.7	0.1
GPT-3 (175B)	3.4	1.1	0.1	1.7	17.7	0.1
GPT-J (All En)	24.3	27.0	23.9	23.3	23.1	23.6
GPT-3 (All En)	24.7	27.2	26.1	24.9	23.6	24.0

Toolformer still lags behind the much larger GPT-3 (175B) model. This is likely due to both the simplicity of our search engine (in many cases, it returns results that are clearly not a good match for a given query) and the inability of Toolformer to *interact* with it, e.g., by reformulating its query if results are not helpful or by browsing through multiple of the top results.

Can small models effectively use tools?







Limitations

- It cannot reliably chain tools together.
- It cannot interactively explore a tool.
- Tool use is sensitive to wording.
- Creating training examples can be inefficient.
- It does not consider tool costs.